Project 2026-16
Public Safety Alerts Aggregator & Explorer (Client Narrative Brief)

What I want here is a system that helps someone answer a simple but important question: **"What public safety alerts are relevant to me right now, and where are they happening?"** In practice, people bounce between multiple websites and feeds depending on the hazard—fire warnings, severe weather, flood advice, road incidents—and each agency presents information differently. During a real event, that fragmentation wastes time and creates confusion, especially when people are trying to make decisions quickly or coordinate with family, colleagues, or students.

At CSU we have staff and students spread across multiple locations, including regional areas where fires, storms, floods, and road disruptions can become very real, very quickly. I'm not trying to replace official emergency services websites. I'm not trying to become an alerting authority. What I want is a **single system** that collects publicly available alerts from free online sources, stores them in a database so they can be queried and analysed, and provides a **web interface** where users can search, filter, and explore alerts—optionally on a map—and, where feasible, subscribe to alert categories and regions they care about.

This project runs across **two sessions**. In Session 1, I want a **manageable prototype** that proves the pipeline end-to-end: collect alerts → normalise them → store them in MySQL → present them in a working web UI with search and filters. In Session 2, I want that prototype expanded into a more robust system: more sources, better de-duplication and update handling, stronger geographic handling for mapping, and a more complete subscription/notification workflow.

I want technology choices kept flexible. Students should choose their stack. But the system is expected to include at least:

- **regular data collection** (RSS/API/permitted scraping)
- **a MySQL database**
- **a backend service** (API endpoints for the web UI)
- **a web front end** for exploration and filtering
- basic logging and safety (rate limiting, input validation, sensible error handling)

Session 1: A manageable prototype with a working web explorer

For the first session, I want the team to focus on doing a smaller set of things well and proving the whole loop works.

To keep it achievable, I'm happy for the prototype to begin with **two or three sources**—as long as they are genuine public alert feeds and cover at least two different categories. Examples might include Bureau of Meteorology warnings, NSW RFS fire warnings, SES incidents, or a road incident feed. The important point is that the system demonstrates it can handle **multiple sources with different formats**, and turn them into a consistent internal structure.

When I open the web interface, I want it to feel like an alert explorer. I should be able to see a list of current alerts and filter them at least by:

- **category** (e.g., weather, fire, flood, road/incident)
- **region** (whatever the source provides—area name, district, LGA label, etc.)
- **status** (active vs expired, or current vs historical)

If I click an alert, I want a detail page that makes it obvious:

- who issued it (source/agency)
- when it was issued and last updated (where available)
- what area it affects
- what the alert says (description text)
- a link back to the official source page if one exists

I'm not asking for perfect mapping in Session 1, because not all feeds provide clean coordinates. But I do want the system to store whatever location information is available and to make a sensible start. If coordinates exist, store them. If only a region name exists, store that and use it for filtering. A basic "map view" is optional at this stage; if you include it, I'd rather it be simple and honest than flashy and misleading.

Behind the scenes, Session 1 must prove that the system is not just "live-fetching and dumping." I want:

- alerts stored in **MySQL**, with enough structure to support filtering and querying
- a collection process that can run **on a schedule or on demand**
- basic de-duplication so repeated ingestion doesn't create endless duplicates
- logs of ingestion runs so it's clear what happened and when

Subscriptions and notifications can be lightweight in Session 1. If you include them, a mock "notification outbox" is fine, as long as it demonstrates the concept: users can subscribe to a category/region and the system can determine which alerts would trigger a notification.

If you can demonstrate a working web UI backed by an API and database—where multiple sources are collected, stored, and explored via search and filters—then Session 1 has achieved what I want.

Session 2: Expanding into a robust aggregator with better location handling and subscriptions

In the second session, I want the system to mature into something that behaves reliably under more realistic conditions and offers more value.

This is where the system should expand to support more sources without becoming fragile. I want the ingestion side to be respectful and stable:

- rate limiting and sensible scheduling
- resilience to temporary failures

- clear logging so issues can be diagnosed
- a design that makes it easy to add a new feed adapter

Session 2 is also where normalisation and de-duplication become much more important. Agencies describe severity and categories differently. The same incident can be updated multiple times or reissued. I want a clear approach so that:

- the system doesn't fill with duplicate alerts
- updates are treated as updates (and ideally tracked as history)
- users can trust that "current alerts" really are current

This is also where the web interface should become more useful. Filtering should become richer—severity thresholds if available, time windows, and clearer region selection. If mapping is included, it should become more than just an extra tab: it should genuinely help users understand where an alert applies. Where feeds provide coordinates or polygons, the system should use them. Where they don't, the interface should avoid giving a false impression of precision.

Subscriptions and notifications should also become more complete in Session 2. I want users to be able to express preferences such as:

- categories they care about
- regions they care about
- minimum severity (if provided by source)
- notification style (immediate vs summary)

You don't need to implement SMS. Email is acceptable. A robust mock outbox is acceptable. But in Session 2, the system should be able to show *why* a notification was generated (which subscription matched which alert).

Finally, Session 2 is where I expect more "operational" features:

- last successful refresh time
- ingestion run history
- source health indicators (even if simple)
- basic uptime/health checks for the backend API

This is what makes the system feel maintainable rather than purely demonstrative.

What success looks like to me

If you do this well, Session 1 will feel like a clean end-to-end prototype: public feeds collected and stored in MySQL, a backend API exposing search/filter endpoints, and a web interface that lets users browse and filter current alerts and open clear detail views with attribution.

By the end of Session 2, it should feel like a robust alert explorer: broader feed coverage, consistent normalisation, sensible de-duplication and update handling, a map view that is

accurate where data allows, subscriptions/notifications that behave predictably, and operational logging that proves the system can run over time.

In short, I want a system that prioritises clarity and trust. In an emergency, people don't need cleverness—they need a single place where alerts are current, consistent, and easy to interpret.